Lesson 1
MLOps Foundations

Willis College
BUSINESS | TECHNOLOGY | HEALTHCARE

Overview

- **Core Concepts in MLOps**
- **Tools and Technologies in MLOps**
- **Foundational Skills**
  - Bash and the Linux Command Line
  - Cloud Shell Development Environments
  - Cloud Computing Foundations

Core Concepts in MLOps

What is MLOps?

- **MLOps** is a combination of **Machine Learning (ML)** and **DevOps** practices.
- **Purpose:** To manage and automate the entire ML lifecycle in production environments.
- Ensures that models are **scalable, reproducible**, and **maintainable**.
- Addresses common challenges like **data drift** and **model retraining**.

Why MLOps?

- **Growing Need:** As ML becomes central to business, companies need efficient, reliable ML systems.
- MLOps allows for **continuous integration** of models with new data.
- **Key Benefit:** Reduces manual intervention and improves deployment speed.
- Allows for **scalable solutions** that meet business needs in real-time.

Rise of the Machine Learning Engineer

- **ML engineers** bridge research and deployment, creating production-ready models.
- **Demand** for these roles has surged with the growth of ML.
- Key skills include **cloud expertise**, **data management**, and **automation**.
- ML engineers are pivotal in bringing value to AI investments in organizations.

Cloud	191K
AWS	83K
Azure	51K
GCP	21K
Data engineer	70K
Machine learning engineer	55K
Data scientist	20K

LinkedIn Job openings 2020

Figure 1. Machine learning jobs (Source: textbook,Figure 1-1)

Certification Pathways for ML Engineers

- **AWS Certified ML Specialist:**
  - Emphasis on **model deployment**, **optimization**.
  - Skills in **AWS SageMaker** and other cloud-native ML tools.
- **Google Professional ML Engineer:**
  - Focus on **ML pipeline design** and monitoring.
  - Builds solutions to solve business challenges.
- **Azure Data Scientist Associate**:
  - Applies data science and ML within Azure's ecosystem.
  - Certifications validate **ML engineering** skills required in cloud environments.

DevOps and MLOps Intersection

- **DevOps**: Automation, CI/CD, and collaboration across development and operations.
- **MLOps**: Extends DevOps principles to ML, covering data, model training, and deployment.
- Both prioritize **speed**, **reliability**, and **scalability** in production systems.
- Shared goals: Streamline and automate deployment, reduce errors, and improve efficiency.

Continuous Integration (CI)

- **CI** allows for frequent code integration and automated testing.
- Detects errors quickly, maintaining code quality and consistency.
- **Popular Tools**: Jenkins, GitHub Actions, GitLab CI/CD.
- CI minimizes integration challenges and keeps code ready for production.

Continuous Delivery (CD)

- **CD** automates the deployment pipeline, ensuring code is ready for production.
- **Infrastructure as Code (IaC)** allows consistent, repeatable environment setups.
- **Reduces Errors**: Minimizes human error in deployment.
- Enables fast, reliable releases with consistent code quality.

MLOps Example

- Setting up a Python project with essential files for **MLOps:**

```
Makefile              # automates tasks
requirements.txt      # specifies dependencies
hello.py              # main Python code
test_hello.py         # unit tests

```

- The Makefile can automate installation, testing, and linting:

```
install:
    pip install --upgrade pip &&\
    pip install -r requirements.txt

lint:
    pylint --disable=R,C hello.py

test:
    python -m pytest -vv --cov=hello test_hello.py

```

CICD Example

- Example GitHub Actions workflow for a CI/CD pipeline.

```yaml
name: ML Model Pipeline
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
      - name: Run Tests
        run: |
          pytest --cov

```

MLOps Pipeline Overview

- **Data Collection:** Ingest and preprocess data for model training.
- **Model Training**: Create and tune ML models for optimal performance.
- **Model Deployment:** Transition models to production for real-world use.
- **Monitoring and Maintenance**: Track model performance and update as needed.

Cloud Native ML Platforms

- Leading platforms: **AWS SageMaker**, **Google Cloud AI Platform**, **Azure ML Studio**.
- Provide end-to-end support for model development, deployment, and scaling.
- Integrated with **big data** resources and **high-performance computing**.
- Allow easy scaling and management of ML workloads.

Tools and Technologies in MLOps

Cloud Native ML Platforms

- Leading platforms: **AWS SageMaker**, **Google Cloud AI Platform**, **Azure ML Studio**.
- Provide end-to-end support for model development, deployment, and scaling.
- Integrated with **big data** resources and **high-performance computing**.
- Allow easy scaling and management of ML workloads.

Containerization in ML

- **Containers** package code and dependencies for consistent deployment across environments.
- Tools: **Docker** (for containers), **Kubernetes** (for container orchestration).
- **Portability**: Containers simplify deployment across diverse environments.
- Enables isolated environments, preventing dependency conflicts.
- Docker example:

```
FROM python:3.8
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Serverless Technology in ML

- **Serverless Computing** abstracts infrastructure, enabling ML with minimal server management.
- Examples: **AWS Lambda**, **Google Cloud Functions**, **Azure Functions**.
- **Cost Efficiency**: Pay only for the compute time used, ideal for intermittent ML tasks.
- Facilitates scalable ML, especially in event-driven architectures.

Specialized Hardware for ML

- Hardware such as **GPUs**, **TPUs**, and specialized processors accelerate performance.
- Essential for high-throughput tasks like **deep learning**.
- Balance **cost** and **speed** for optimal performance.

Monitoring and Instrumentation

- **DataOps and Data Engineering**
  - **DataOps**: Automates data pipelines and ETL processes.
  - **Data Engineering**: Prepares data for ML pipelines.
  - Tools include **Apache Airflow** and **AWS Glue**.
- **Monitoring and Instrumentation**
  - Tools: **DataDog**, **New Relic**, and **Stackdriver**.
  - Monitors **data drift** and **model degradation**.

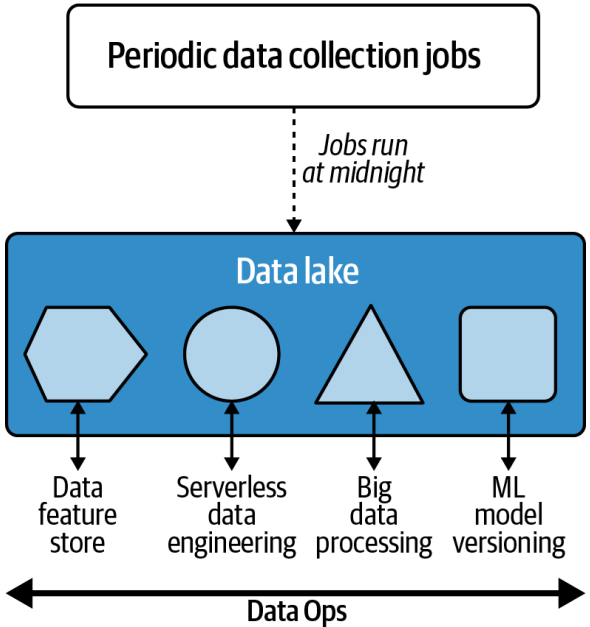

Figure 2. Data engineering with a cloud data lake (Source: Figure 1-8, textbook.)

Specialized Hardware for ML

- Hardware such as **GPUs**, **TPUs**, and specialized processors accelerate performance.
- Essential for high-throughput tasks like **deep learning**.
- Balance **cost** and **speed** for optimal performance.

Foundational Skills

Bash and the Linux Command Line

- **Linux Command Line** skills are crucial for working in cloud environments.
- Common commands: **ls**, **cd**, **pwd**, and scripting with **bash**.
- Efficient for **automation** and interacting with cloud environments.
- **Cloud Shell Development**
  - **Cloud Shells** provide development environments directly on cloud providers.
  - **AWS CloudShell** and **AWS Cloud9** are key tools.
  - Secure, scalable, and fully managed.

Bash and the Linux Command Line

- **Linux Command Line** skills are crucial for working in cloud environments.
- Common commands: **ls**, **cd**, **pwd**, and scripting with **bash**.
- Efficient for **automation** and interacting with cloud environments.
- **Cloud Shell Development**
  - **Cloud Shells** provide development environments directly on cloud providers.
  - **AWS CloudShell** and **AWS Cloud9** are key tools.
  - Secure, scalable, and fully managed.

Minimalistic Python Tutorial

- Core elements of Python: **functions** and **statements**.

```python
def add(x, y):
    """Simple addition"""
    return x + y
print(add(1, 2))

```

- **Data Cleaning** and **scaling** for improving model performance.

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

Model Training and Hyperparameter Tuning

- **Hyperparameter Tuning**: Optimizes model parameters.

```
from sklearn.model_selection import GridSearchCV

params = {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']}
svc = SVC()
clf = GridSearchCV(svc, param_grid=params)
clf.fit(X_train, y_train)

```

- **Optimization and Gradient Descent**
  - **Gradient Descent** is key for finding model parameters that minimize errors.
  - Aids in improving model predictions.

Data Versioning with DVC

- **DVC** helps track data changes alongside code.

```
# Initialize DVC
dvc init

# Add data
dvc add data.csv
git commit -m "Track data with DVC"

```

Model Deployment with Flask

- Deploy ML models as REST APIs using **Flask**.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json(force=True)
    return jsonify(prediction=model.predict(data))
app.run(debug=True)

```

Testing in MLOps

- **Unit Testing**: Validates model components.
- **Integration Testing**: Ensures compatibility within the system.
- Improves model reliability in production.
- **Security and Scalability in MLOps**
  - **Security**: Protects data and models from unauthorized access.
  - **Scalability**: Ensures models handle growing data and user load.

References

- Gift, N., & Deza, A. (2021). Practical MLOps. " O'Reilly Media, Inc.". pages 21-86